

NetAuditAI: Architecture (2-page brief)

SIH 2026 · Problem Statement 26155 · AI-Driven Multi-Vendor Network Security Compliance Auditor (NTRO). Full design: architecture.md.
Setup and limits: README.

1. The idea

The problem statement names the trap: a hard-coded command library "becomes obsolete as vendors release firmware updates", and parsers "fail because they cannot predict the configuration structures of newly acquired hardware". A parser per vendor is a losing race against Palo Alto, Cisco, Arista, SONiC, Azure NSGs and whatever is bought next quarter.

NetAuditAI inverts it. Instead of *N* parsers it asks **23 security questions** (is Telnet enabled? which SSH version? an idle timeout, a remote syslog, an any-to-any rule?) and extracts only the **security facts** those questions need. A fact is one cited, typed statement: `mgmt.ssh.version = 1`, line 42, assurance confirmed. Controls read facts and never vendor syntax, so **every control runs on every configuration**.

The other half of the design is honesty about evidence. Each fact carries an **assurance**, a verdict inherits the weakest assurance it cites, and only decisive facts are scored. **Absence is never a PASS**: a setting nobody configured is NOT_CONFIGURED, not compliant. In an audit, "I could not determine this" beats a confident wrong answer. Absence becomes a FAIL only for settings no device ships with (AAA, remote syslog, banner, NTP), and only when learned knowledge understands the file's dialect and knows how that dialect would write the missing line.

2. Pipeline

```
Ingest ... uploaded file (single or bulk, CLI text or JSON export)
... or read from the live device over SSH (Netmiko / NAPALM, 12 platforms, read-only)
-> UTF-8, <= 2 MB, JSON and Terraform flattened to one statement per rule, secrets redacted before any AI call
-> Vendor identification: fingerprint + grammar coverage (a look-alike is never "confirmed")
-> Confirmed vendor ..... dedicated parser (Cisco IOS, FortiGate)
-> Anything else ..... generic tokenizer (braces, indentation, set-style, /menu paths)
    -> recognizers, 226 shipped + whatever was taught [pattern recognition]
    -> lexicon heuristics [provisional]
    -> AI judge on what is still undecided [budgeted, must cite]
-> Security Baseline Model: facts with value, scope, evidence lines, assurance
-> Deviation analysis: 23 controls -> PASS / FAIL / UNKNOWN / NOT_CONFIGURED / N_A
-> Posture (pass / decided) + Coverage (decided / applicable) + critical controls not assessed
-> Framework views: NIST SP 800-53 Rev. 5 · CIS · DISA NDM SRG · ISO/IEC 27001:2022 Annex A
-> Remediation: vendor CLI for confirmed vendors, verified by re-parse; candidates simulated on a copy
-> Per-device PDF report (ReportLab)
```

Live collection is only a fetch in front of that same pipeline, so a collected device and an uploaded file give the same result, redaction and AI rules. It is fenced in: a host is refused unless every address it resolves to is private (link-local by name, that being the cloud metadata endpoint), credentials last one session and are never stored or logged, and nothing is ever written back to a device.

3. The Security Baseline Model

The vendor-neutral schema is a fact vocabulary of **23 predicates** (`app/facts/predicates.py`), among them `mgmt.remote_access.protocol_enabled[telnet]`, `mgmt.ssh.version`, `mgmt.session.idle_timeout` (minutes), `auth.password.storage`, `boundary.policy.permit_any`. Cisco transport input telnet, Junos services { telnet; }, EXOS enable telnet and Gaia set telnet-server enabled true all land on one fact. Structured exports (AWS security groups, Azure NSGs, GCP firewall rules, SONiC config_db.json) and Terraform files are flattened first, so each rule object reads as a single statement. Deviation analysis then compares facts to the chosen benchmark, exactly as the problem statement describes: parsed `ssh_version` against what CIS requires.

4. Dynamic adaptation: the training loop

When the engine meets syntax it cannot read, the line surfaces in the **Interactive Training Interface** beside the control it could answer. The administrator says what it means in plain words ("this sets the idle timeout"). No regex, no code: the system drafts a **recognizer**, a typed template such as `idle-timeout {duration:min}` scoped to its block, with {neg} so one entry reads both `telnet server` and `telnet server`.

Safety gates (`validate_recognizer`) reject anything that would match too much: two keywords, stated polarity or a value table, a unit for durations, and no secret is ever stored. The recognizer saves to SQLite or Postgres and answers the **very next scan**: decisive, deterministic, no AI call, **no redeployment**, reversible later.

Shipped seed knowledge is the same mechanism reviewed in Git rather than taught at runtime: **226 recognizers** across Juniper Junos, Palo Alto PAN-OS, Arista EOS, Huawei VRP, HPE Aruba AOS-CX, Check Point Gaia, Extreme EXOS, MikroTik RouterOS, Cisco NX-OS, ASA and IOS-XR, and AWS security groups. A fresh deployment reads those dialects before anyone teaches it.

5. Where AI is used, and where it is not

Step	AI?	Why
Deciding compliance	Never	Deterministic Python: reproducible, auditable, identical every run
Reading an unknown dialect	Optional escalation	The judge sees only redacted scopes of undecided controls, must cite line numbers, and every citation is re-verified against the file. Its answer stays a proposal until a human confirms it
Explaining a finding	Optional	Plain-language commentary in the finding drawer, from redacted evidence
Asking about a scan	Optional	An assistant answers from that scan's already-redacted results, told the verdicts as facts and that undecided is not failure
Candidate remediation	Optional	Proposed commands are simulated on a copy and re-evaluated before confirmation

Pattern recognition does the heavy lifting; the LLM is a bounded escalation that proposes but never decides. Redaction (`app/ai/redaction.py`) runs before every AI call and every response quoting configuration: passwords, hashes, PSKs and community strings become typed placeholders like `<SECRET:type7>`, so a weak-encoding control still knows the password is Type 7 without the value leaving the process.

6. The five required capabilities

Required	How it is met
Unified ingestion	One dashboard: single or bulk upload, CLI text or JSON export, or pull straight from devices over SSH

Required	How it is met
AI-powered training module	The Interactive Training Interface above: low-code, plain-words mapping, live on the next scan, no redeployment
Multi-framework engine	NIST SP 800-53 Rev. 5, CIS, DISA NDM SRG, ISO/IEC 27001:2022 Annex A. One benchmark chosen at upload or all four reported; every control runs either way
Actionable intelligence and PDF report	Per device: identification (hostname, vendor, OS version, model, serial when the configuration states them), PASS/FAIL with severity and cited lines, framework mapping, device-specific remediation CLI with its verification, and every undecided control with what it needs
Vendor-agnostic scalability	A new vendor is taught, not coded. A new structured format is flattened automatically. A new device to collect from is one platform entry. A new framework is a mapping; a new question is one predicate plus one control

7. Technology and scale

Python 3.10, FastAPI, ReportLab, Netmiko (NAPALM optional), SQLite or Postgres via psycopg, Groq LLM API (optional), React 19 and Vite with no runtime UI dependencies. **1588 backend tests** (pytest) and **149 frontend** (Vitest). Everything runs without AI; AI only raises coverage.

Honest scope: two vendors have dedicated parsers (Cisco IOS, FortiGate). Everything else is read generically, with eleven dialects and AWS security groups already answering decisively from shipped recognizers. Any other configuration is still ingested, tokenized and evaluated; what cannot be decided is reported as undecided rather than guessed, until somebody teaches the line that settles it.